



Introduction to SQL Window Functions

PARTITION BY, Window Functions, and DENSE_RANK()

Dr. Sumaiya Tabassum Nimi

Assistant Professor, Dept. of ECE — North South University

Introduction

In SQL, **window functions** let you perform calculations *across a set of rows* that are related to the current row *without collapsing them into a single output*.

This is different from **GROUP BY**, where rows are aggregated into one row per group. Window functions keep all rows and allow advanced analytics:

- Ranking (e.g., TOP employee per department)
- Running totals
- Moving averages
- Comparisons within groups

Window functions are widely used in:

Business analytics, dashboards, finance, recommendation systems, and logs analysis.

Basic Syntax of a Window Function

A window function uses the following general structure:

```
function() OVER (PARTITION BY ... ORDER BY ...)
```

Example:

```
DENSE_RANK() OVER (PARTITION BY Department ORDER BY Salary DESC)
```

Here's what each component does:

- **PARTITION BY**: divides rows into groups (like **GROUP BY**, but does NOT collapse rows).
- **ORDER BY**: determines how rows are ordered within each partition.
- **Window Function**: performs ranking, running totals, etc., over the window.

Understanding PARTITION BY

Intuition

Think of PARTITION BY as creating “mini-tables” inside your query. The window function then runs *within each mini-table*.

Example scenario: You have employee salaries and want to rank employees *within their department*:

Engineering: Rank employees by salary
HR: Rank employees by salary
Finance: Rank employees by salary

Each department becomes its own partition.

DENSE_RANK(): A Beginner-Friendly Ranking Function

What is DENSE_RANK()?

DENSE_RANK() assigns ranks to rows within each partition, but:

- Ties = same rank
- No gaps in the ranking sequence

Example:

If salaries in a department are:

100k, 95k, 95k, 80k

Then:

Salary	Rank
100k	1
95k	2
95k	2
80k	3

Notice rank 3 appears (NO missing rank 3).

Example Dataset

Consider this simple table:

Employee	Department	Salary
A	Engineering	90000
B	Engineering	85000
C	Engineering	85000
D	HR	60000
E	HR	55000

Example Query Using DENSE_RANK()

Goal: Rank employees inside each department by salary.

```
SELECT
    Employee,
    Department,
    Salary,
    DENSE_RANK() OVER (
        PARTITION BY Department
        ORDER BY Salary DESC
    ) AS SalaryRank
FROM Employees;
```

Explanation:

- **PARTITION BY Department:** creates groups (Engineering, HR)
- **ORDER BY Salary DESC:** highest salary gets rank 1

Result:

Employee	Department	Salary	SalaryRank
A	Engineering	90000	1
B	Engineering	85000	2
C	Engineering	85000	2
D	HR	60000	1
E	HR	55000	2

Why Use Window Functions Instead of GROUP BY?

GROUP BY removes row-level detail:

```
SELECT Department, MAX(Salary)
FROM Employees
GROUP BY Department;
```

This gives:

Engineering – 90000
HR – 60000

All other employee records vanish.

Window functions:

- keep all rows
- add new analytical columns
- allow richer insights

Summary

- Window functions perform calculations across related rows.
- **PARTITION BY** divides data into groups.
- **DENSE_RANK()** creates rankings without gaps.
- Window functions are essential in data analytics, dashboards, finance, and reporting.